TRABAJO PRACTICO Nº 3

Integrador (OO, C++, red de datos, Python/otro)

A. Elementos necesarios

Guías de clase.

Trabajos Prácticos Nº 1 a 2.

Documentos relacionados (ver anexo 2)

Dispositivos y plataformas de desarrollo.

B. Entrega y Evaluación

La fecha de entrega del informe y de la presentación grupal habitualmente se indica al principio del ciclo, pero puede ser ajustada por mutuo acuerdo en clase.

Esta presentación es realizada por el/los autor/es, con exposición pública y posterior coloquio grupal e individual de ampliación y defensa del trabajo, abordando aspectos teórico-prácticos tomando como base la implementación realizada.

C. Actividades Generales

1. A partir del enunciado de la consigna específica, de los temas especiales involucrados (gestión de archivos, gestión de memoria, subprocesos, comunicaciones mediante protocolos y/o mecanismos de red) y de los temas opcionales de su interés particular (interfaces gráficas de usuario, interacción con teclado y/o mouse, manipulación de gráficos, sonido, animaciones, Web, uso de móvil, etc.), investigue sobre las capacidades de los lenguajes C++ y Python para manejar los mismos.

2. Puede acordar con el profesor los detalles que no estén especificados con claridad en este enunciado.

3. Proponga e implemente una Solución Orientada a Objetos, aplicando los conceptos fundamentales desarrollados: Clases, Objetos, Herencia, Agregación, Polimorfismo, Constructores/Destructores, Sobrecarga de Métodos, Sobrecarga de Operadores, Control de errores mediante Clases de Excepción, Contenedores de Objetos, gestión de Streams de E/S, etc.

4. Una vez finalizada la implementación, confeccione un informe, siguiendo un esquema similar al indicado en el anexo 1 (puede tomar como base el utilizado en las entregas anteriores y ajustarlo convenientemente).

5. Elabore un video breve (10'-15'). En el mismo use de 5' a 7' para explicar sintéticamente su solución, preferentemente basada en gráficos y/o esquemas asociados tanto conceptuales como de la implementación realizada; y en el tiempo restante muestre el funcionamiento/uso de su aplicación.

6. Realice la entrega de los archivos resultantes en formato comprimido (código fuente del proyecto, incluyendo librerías de terceros usadas, ejecutables, informe y videos), en la actividad de entrega habilitada en el aula virtual (o envíe por correo al profesor si se indicara explícitamente).

D. Consigna específica

1. Dominio del problema

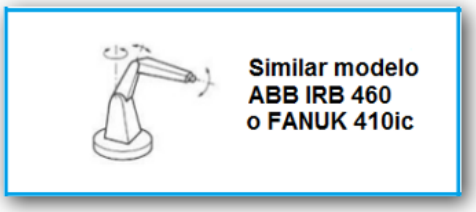
- Programación Orientada a Objetos para Manipulación Remota de un Robot.

2. Objetivo

- Desarrollar un aplicativo para controlar un robot de 3 grados de libertad con efector final, conforme a los requerimientos provistos.

3. Robot específico

- Tipo: robot 3DF.
- Configuración: RRR



4. Requerimientos del Software

- Aspectos generales
  - El aplicativo/sistema a desarrollar, debe estar representado e implementado usando paradigma orientado a objetos en todas sus capas y puntos de despliegue.
  - Debe diseñarse separando la capas de modelo propia del dominio de la capa de presentación, de manera que la solución pueda usarse independientemente de que se use interfaz gráfica o no.
  - Efector final: simple (puede considerarlo tipo pinza, sólo con acciones del tipo activar/desactivar).
  - Dimensiones (extremas): anchura=A mm, profundidad=B mm, altura=C mm. y Velocidades máximas: lineal=VL mm/s en Z y en el plano XY. Estos parámetros son necesarios para limitar los valores que pueda introducir el operador. Consultar la documentación anexa disponible en el aula virtual.
  - El controlador del robot recibe órdenes en formato G-Code y devuelve mensajes con información de estado. No debe desarrollarse el firmware correspondiente, ya que el mismo se provee, adaptado para simulación, usando tecnología Arduino. Disponible en el aula virtual.
  - Dicho controlador, conoce los parámetros geométricos y cinemáticos del robot. Realiza un control parcial de la viabilidad de cada orden recibida, retornando un breve mensaje o código si corresponde. Consultar la documentación anexa disponible en el aula virtual
  - Los mensajes (ordenes y datos) enviados entre cliente y servidor deben manipularse mediante objetos de dominio específicos (preferentemente sólo 1, con la misma definición en cada extremo), responsables de la serialización y deserialización correspondiente.

- Comportamiento del lado servidor:
  - Los módulos del lado servidor deben estar desarrollados en C++.

- Mantener un archivo con el registro del trabajo (log acumulativo) realizado por el servidor. Este archivo responde a un formato CSV estándar. La información mantenida en caso de peticiones tiene: marca de tiempo de cada petición al servidor, detalle de la petición, usuario que realizó la misma, nodo desde donde se conectó, código de la respuesta dada. En caso de funcionamiento general tiene: marca de tiempo en que se produjo el evento en el lado servidor, módulo, mensaje con el detalle de lo ocurrido.
- Mantener una tabla en base de datos (MySQL, SQLite, etc) con el id, nombre, clave de acceso y privilegios de cada lo usuario autorizado. Sólo 1 de ellos es administrador y tiene acceso a los servicios prioritarios, el resto tiene algunas restricciones. Queda a criterio del equipo definir más niveles.
- Ofrecer servicios vía XML-RPC, que permitan a usuarios remotos o locales:
    - Validar cada petición mediante usuario y clave.
    - Conectar/desconectar al Robot, vinculado al servidor mediante una comunicación serie (sólo el administrador).
    - Habilitar/deshabilitar el acceso remoto de otros usuarios
    - Activación/Desactivación de los motores del robot.
    - Listar los comandos disponibles de control (de ser posible con un breve mensaje de ayuda al operador, ejemplificando la sintaxis o límites, etc).
    - Ofrecer al operador un reporte en consola de información general (conteniendo el estado de la conexión, la posición y el estado de actividad actual, el momento en que se inició la actividad, un listado con el detalle de las órdenes y la cantidad de ellas que han sido solicitadas desde la ultima conexión, detallando las que produjeron un error). Cada usuario ve su propio detalle.
    - Ofrecer al administrador un reporte en consola con el mismo tipo de información anterior, pero sin restricciones, incluyendo la posibilidad de filtrar por al menos 2 criterios (los acuerda el equipo de desarrollo).
    - Ofrecer al administrador un reporte en consola con detalles del log de trabajo del servidor, incluyendo la posibilidad de filtrar por al menos 2 criterios (los acuerda el equipo de desarrollo).
    - Subir/almacenar archivos con código G-code, que contengan tareas (secuencias) completas de trabajo del robot.
    - Seleccionar un modo de trabajo (manual o automático, absoluto o relativo)
    - [modo manual]: Realizar movimiento del efector a su “posición de origen/descanso”.
    - [modo manual]: Mover el efector especificando la velocidad y la posición final.
    - [modo manual]: Mover el efector especificando sólo la posición final.
    - [modo manual]: Activar/Desactivar el efector final.
    - [modo manual]: Aprender una trayectoria paso a paso, con almacenamiento en archivos de texto (cuyo nombre es provisto por el operador), de las órdenes correspondientes en formato G-Code.

        - [modo automático]: Carga y ejecución de un archivo seleccionado conteniendo una secuencia de trabajo específica, en formato G-code. El operador debe poder elegir el archivo a usar, siempre que sea de su propiedad. El administrador, puede seleccionar cualquiera
    - Ofrecer un panel de control del robot, mediante consola tipo CLI, sólo para el administrador, que permita realizar las funciones anteriores. Para acceder a la interfaz, debe validarse al usuario.
- Comportamiento del lado Cliente:
    - Los módulos del lado cliente pueden estar desarrollados en un lenguaje diferente a C++. El mismo puede ser Python u otro de preferencia del equipo de desarrollo.
    - Ofrecer una interfaz de usuario que permita controlar al robot de manera remota, con los mismos servicios que los ofrecidos en la interfaz de consola del lado servidor. Su uso o no depende del perfil de usuario.
    - Opción de menú para subir al servidor, archivos de tareas en formato G-Code y almacenarlos, de modo que puedan ser usados por el robot.
    - [opcional] La interfaz del lado Cliente puede orientarse a una interfaz de usuario gráfica (escritorio, web o móvil).
    - [opcional] Despliegue visual del robot, usando vista 3D o proyecciones. En cualquiera de los casos puede usarse una estructura simplificada del robot, ejes/planos y escala (a su criterio).
    - [opcional] Streaming de video mostrando el comportamiento del Robot usando una cámara remota.
    - [opcional] Efectos de sonido que indiquen el arranque/parada del robot, los cambios de posición del efector final y los momentos de actividad del mismo.
    - [opcional] Generación de alarmas visuales/auditivas cuando se intenten desplazamientos fuera del espacio de trabajo (aproximado).
- Observaciones y restricciones:
    - Defina al menos 2 packages, uno para el cliente y otro para el servidor.
    - El efector final no agrega grados de libertad especiales, más allá de algún desplazamiento fijo en las coordenadas asumidas para el punto de operación.
    - Para simplificar el desarrollo, considere que en modo manual, el robot recibe las órdenes de a una por vez.
    - La entrada de cada posición por parte del operador, se desea que sea en coordenadas (X, Y, Z). Sin embargo, se admite de manera secundaria (opcional) la directa de comandos G-Code.
    - Considere las situaciones de concurrencia que pudieran darse y proponga los mecanismos/algoritmos para resolverlas.
    - Otros datos constructivos del robot o plataforma pueden ser indicados durante el desarrollo.
    - Se sugiere partir de la implementación de un prototipo global ejecutable no funcional, e ir mejorándolo en sucesivos ciclos de desarrollo y revisión.

ANEXOS

I. Esquema del Informe

1. Portada (donde consten datos de identificación personales y de contexto)
    a. Identificación de materia
    b. Identificación del trabajo
    c. Identificación de alumno/s
    d. Identificación de docente/s
    e. Identificación de carrera
    f. Identificación de facultad/universidad
    g. Identificación de ciclo lectivo
2. Índice
3. Introducción
    a. Descripción breve del problema
4. Cuerpo
    a. Conceptos/fundamentos/esquema o gráfica descriptiva, con detalles estructurales o funcionales para cada uno de los Temas Especiales agregados (agregue los esquemas o imágenes que estime convenientes para acompañar sus descripciones).
    b. Descripción de componentes software (librerías, frameworks, aplicaciones, etc.) usados para implementar los aspectos anteriores (agregue los esquemas o imágenes que estime convenientes para acompañar sus descripciones).
    c. Descripción/ejemplificación paso a paso del modo de instalación/vinculación/uso, aplicado a la solución específica
    d. Incluya el modelo OO de lo implementado (tanto del lado servidor como del lado cliente), mediante Diagramas de Clases de diseño (debe ser consistente con la implementación)
    e. Incluya un Diagrama de Secuencia para un escenario completo de trabajo, a partir de un requerimiento de usuario (debe ser consistente con la implementación). Elija uno de complejidad intermedia, no trivial.
    f. Incluya un Diagrama de Actividad, que muestre una perspectiva global del aplicativo Servidor.
    g. Describa los detalles de instalación, configuración y ejecución particulares de la implementación.
    h. Capturas de pantalla mostrando la ejecución del aplicativo, para cada una de las interfaces de entrada/salida propuestas para interacción con el usuario.
    i. Módulo y reporte de Prueba Unitaria para alguno de los siguientes módulos (a elección):
        - Interfaz de línea de comandos, lado servidor.
        - Clase Robot (o su equivalente) perteneciente a la capa de modelo.
        - Gestor de archivo/s
        - Servidor RPC
5. Comentarios y Conclusiones

   a. Comentarios sobre las dificultades enfrentadas y su resolución, justificaciones a la solución propuesta (de diseño, de implementación, de instalación, etc.)
   b. Análisis de ventajas/desventajas de los componentes usadas
   c. Posibles extensiones a la solución dada

6. Referencias
   a. Bibliografía/Sitios web consultados
   b. URLs a los recursos software (tutorial, video, ejemplo, herramienta, etc.) que se hayan usado.

II. Robot y simulador

El desarrollo original pertenece a Florin Tobler, se encuentra en https://www.thingiverse.com/thing:1718984. Luego, el trabajo fue adaptado y continuado por diferentes autores.

Para la propuesta de la este Trabajo Práctico se hizo una extensión del diseño, disponible en https://www.thingiverse.com/thing:3674358

Y una bifurcación del código. Algunos de los cambios se encuentran como aporte a la comunidad RobotArm (https://www.facebook.com/groups/robotarm).

Para su aplicación en este Trabajo Práctico, se derivó en 2 versiones, una para usar como simulador de comportamiento del robot, útil en la fase de desarrollo OO de las interfaces de usuario y otra con el firmware, operativo para usar con el robot físico. Ambos códigos, son provistos en el aula de la asignatura.

El software del simulador es una modificación del código que funciona en el robot real.

Para que el producto resultante de este TP, pueda utilizar el simulador se requiere cargar este último a una placa Arduino MEGA (ya que el robot se controla desde una placa RAMPS 1.4), o bien con limitaciones a una placa Arduino UNO.

En cualquier caso, debe mantenerse la anterior, conectada al puerto serie de la computadora de desarrollo.

Se puede realizar una interacción primaria, a través de un programa monitor del puerto serie (como el incorporado en Arduino IDE).

Si bien se han deshabilitado todas las entradas/salidas del microcontrolador, por precaución se recomienda que la placa no se encuentre conectada a ningún periférico, ni plaqueta auxiliar, y colocar la misma dentro de algún gabinete aislante para evitar cortocircuitos durante el uso.

Las órdenes enviadas al robot/simulador hacen uso de lenguajes habituales en máquinas de control numérico, como son G-Code y M-Code.

Para trabajar en modo simulacion, se ha previsto además, el uso de un NodeMCU v3.

Antes de cargar el firmware a la placa que use para el desarrollo, defina como **true** las macros **SIMULATION** y **USE_ESP8266**, el archivo **config.h**

Para trabajar en modo real (conectado al Robot físico), asigne **false** a las anteriores.

Por defecto, se asume una placa MEGA 2560.

Si usa una Arduino Uno, asigne true a la macro USE_UNO.

Para la interacción a través del canal serie, tenga en cuenta que los mensajes usen final de línea (‘\n’) y retorno de carro (‘\r’).

- Tabla de mensajes de entrada/salida del simulador

En la siguiente tabla se pueden observar los comandos puede recibir el robot durante su operación y las respuestas correspondientes, que serán devueltas por puerto serie.

Tenga en cuenta que, cada comando reconocido como válido y "ejecutado" devuelve OK, además de información adicional. Mientras que un comando inválido, sólo retorna un mensaje.

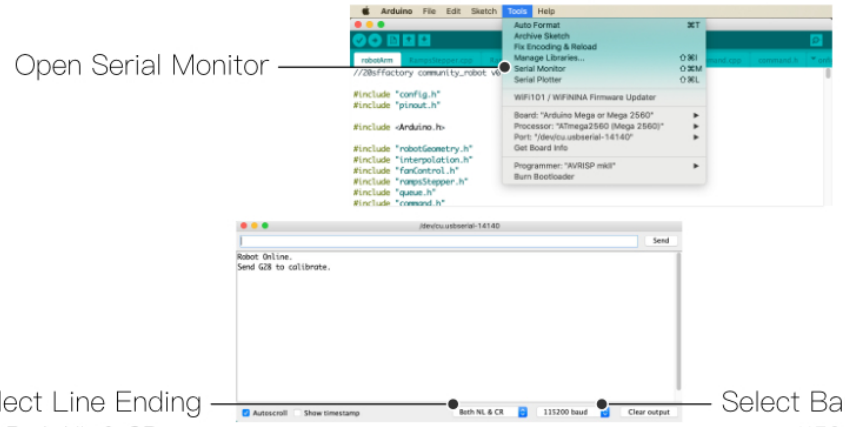
Entrada	Acción	Respuesta
G0	Mover el brazo a la posición (a,b,c)[mm] . El modo de trabajo de las coordenadas es absolutas, por defecto. Siempre, devuelve las coordenadas absolutas luego de realizado el movimiento, independientemente de que se haya realizado un movimiento relativo. La coordenada E, no debería usarse. Hace referencia a otro eje no implementado.	INFO: LINEAR MOVE: [X:0.00 Y:170.00 Z:120.00 E:0.00] En caso de que el punto solicitado este fuera del espacio de trabajo devuelve un error: ERROR: POINT IS OUTSIDE OF WORKSPACE
G1	Idem G0	Idem G0
G4	Modificador de la rampa de inicio/parada en movimientos.	ERROR: SPEED DELAY NOT IMPLEMENTED
G28	Moverse a la posición Homing	INFO: HOMING COMPLETE
G90	Modo de coordenadas absolutas.	INFO: ABSOLUTE MODE ON
G91	Modo de coordenadas relativas.	INFO: RELATIVE MODE ON
G92	Setear la posicion actual como de 0 desplazamiento	INFO: POSITION OFFSET: [X0.00 Y:0.00 Z:0.00 E:0.00] INFO: CURRENT POSITION: [X:0.00 Y:170.00 Z:120.00 E:0.00]
M1	Activar bomba.	ERROR: PUMP ENABLED NOT IMPLEMENTED
M2	Desactivar bomba.	ERROR: PUMP DISABLED NOT IMPLEMENTED
M3	Activar gripper.	INFO: GRIPPER ON
M5	Desactivar gripper.	INFO: GRIPPER OFF
M6	Activar laser.	ERROR: LASER ENABLED NOT IMPLEMENTED
M7	Desactivar laser.	ERROR: LASER DISABLED NOT IMPLEMENTED
M17	Activar motores	INFO: MOTORS ENABLED
M18	Desactivar motores	INFO: MOTORS DISABLED
M106	Activar ventilador.	INFO: FAN ENABLED
M107	Desactivar ventilador.	INFO: FAN DISABLED
M114	Reporte con información general de estado del robot (modo de trabajo, coordenadas de la posición actual, motores y ventilador)	INFO: RELATIVE MODE INFO: CURRENT POSITION: [X:0.00 Y:170.00 Z:120.00 E:0.00] INFO: MOTORS DISABLED INFO: FAN DISABLED
M119	Reporte del estado actual de los finlades de carrera de cada eje.	INFO: ENDSTOP: [X:1 Y:1 Z:1]
Otros		ERROR: COMMAND NOT RECOGNIZED

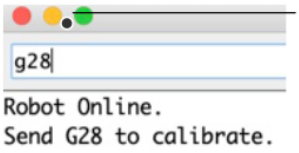
A continuación, otra documentación obtenida del proyecto original, que puede ser de interés.

- Comandos

Command Guide (Arduino IDE)

Arduino IDE Serial Monitor can perform basic single command serial communications.
Suitable for quick testing after firmware upload is completed.





Calibrate Robot

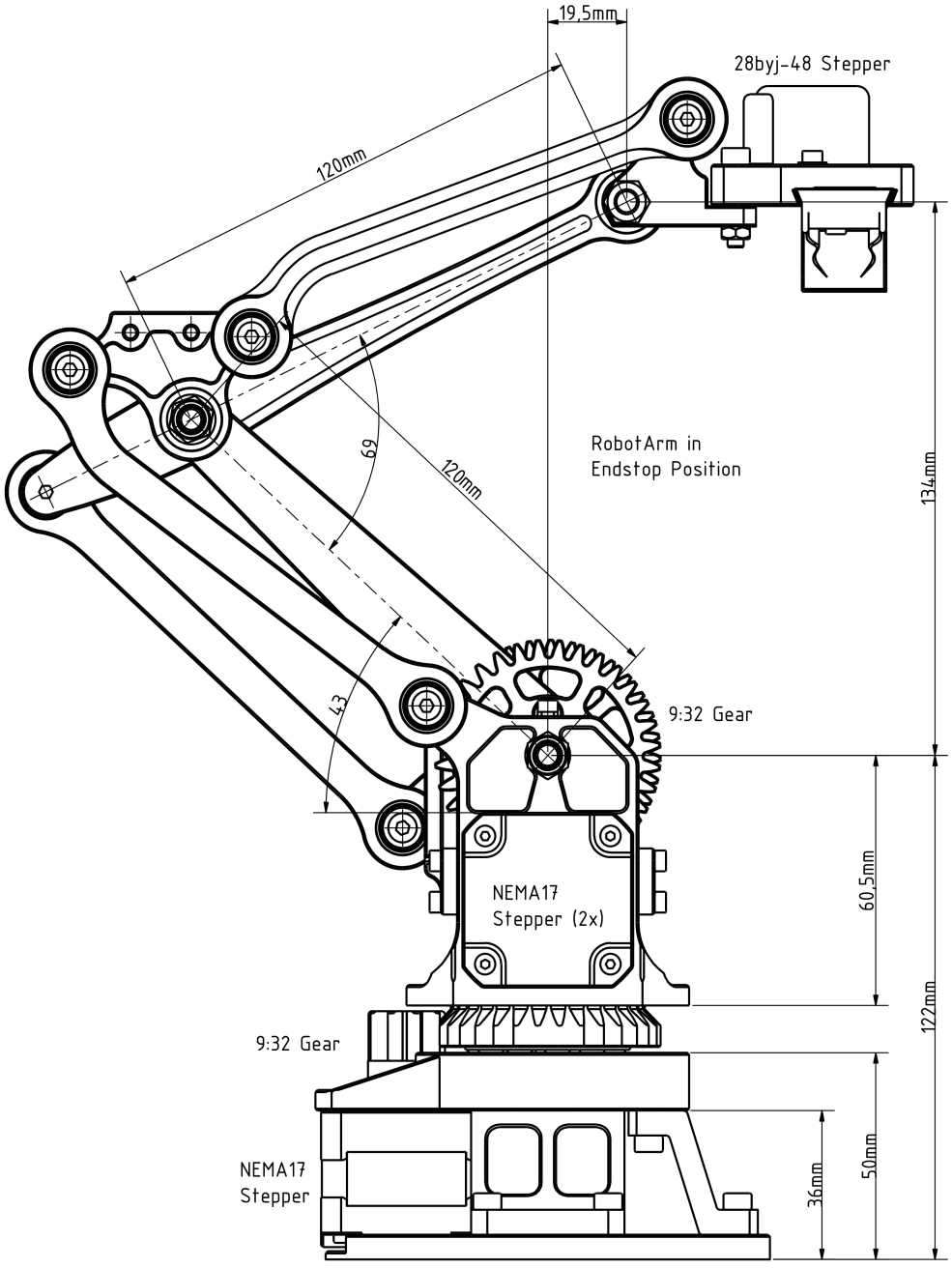
With Endstops:
Send G28
Without Endstops:
Move Arms into Endstop Positions
Rotate Body to Center Position
Send G28

Command List

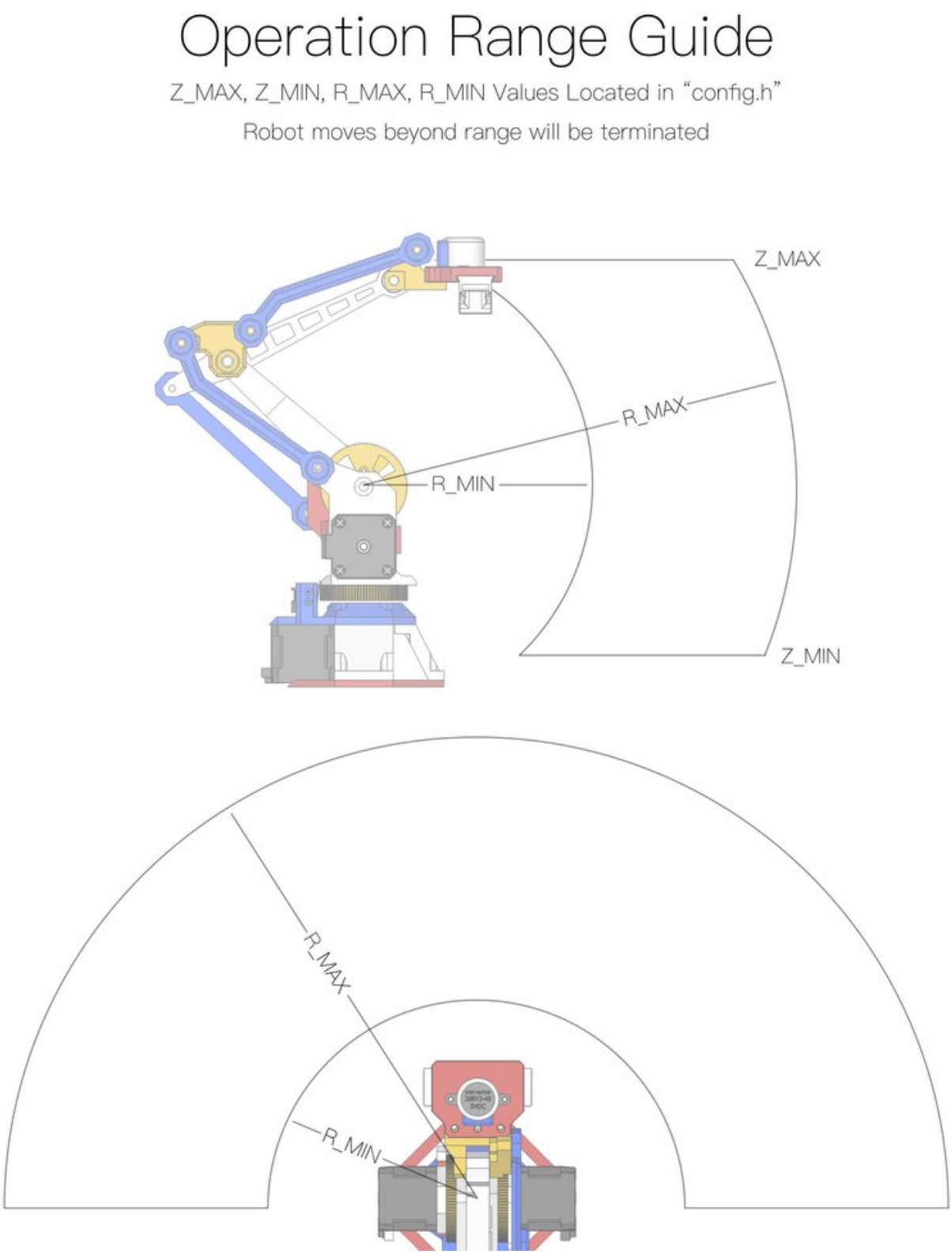
See Cartesian Guide for Coordinate Reference

Linear Move	G0 X<x_mm> Y<y_mm> Z<z_mm> F<mm/s> eg: G0X120Z-10F30	Pump On/Off	M1 / M2
Dwell	G4 S<sec> (waits for <sec> seconds) eg: G4S10	Gripper On/Off	M3 / M5
Homing	G28 (home all motors if homing enabled)	Laser On/Off	M6 / M7
Absolute Mode	G90 (moves in absolute coordinate)	Steppers Enable/Disable	M17 / M18
Relative Mode	G91 (moves in increment value)	Fan Enable/Disable	M106 / M107
Set Position	G92 X<x_mm> Y<y_mm> Z<z_mm> E<mm/s> eg: G92 Z0 (sets current z value to 0) eg: G92 (resets all offset values)	Report Coordinates	M114
		Report Endstop State	M119
Set Speed Curve	M205 S[SPEED_PROFILE] eg: M205 S0 (sets flat speed curve) eg: M206 S2 (sets cosin speed curve)		

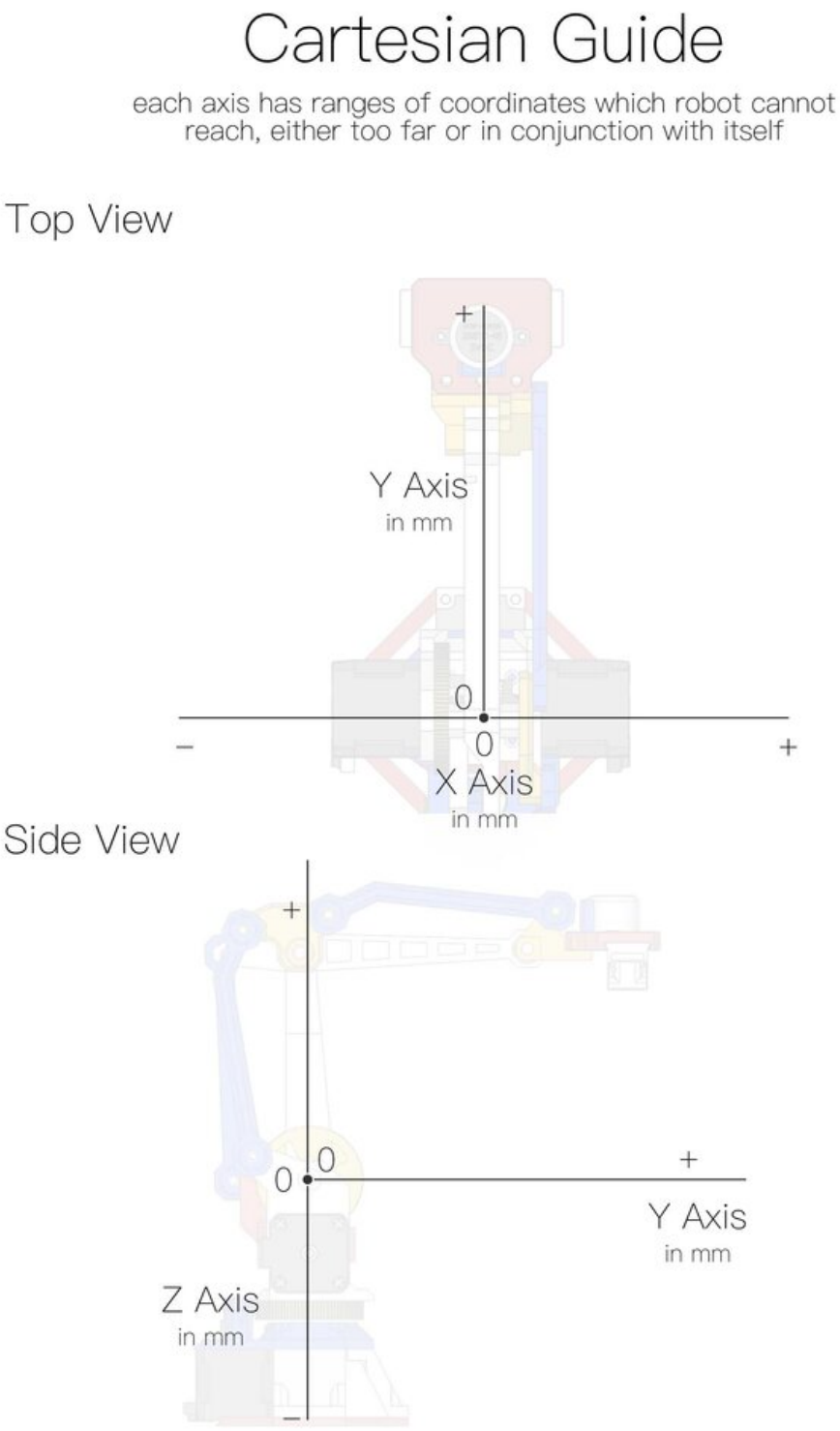
- Dimensiones del Robot:



- Espacio de trabajo:



- Orientación cartesiana



III. Programación de máquinas NC con códigos G&M

La programación nativa de la mayoría de las máquinas de Control Numérico Computarizado (como fresadoras, cortadoras, tornos, impresoras 3D y robots) utiliza, además de algunos lenguajes propietarios y/o tradicionales, un lenguaje de bajo nivel habitualmente llamado G&M.

Los programas de G&M son simples archivos de texto ASCII (sólo mayúsculas, números y signos de puntuación tradicionales, por lo que es muy frecuente que los programas se almacenen y comuniquen usando un formato restringido de 6 bits).

El nombre G&M proviene de que el programa está constituido por instrucciones Generales y Misceláneas.

Si bien en el mundo existen aún diferentes dialectos de programación con códigos G&M, existe una estandarización que promovió la ISO. Esta estandarización fue adoptada por la totalidad de los fabricantes industriales de CNC (tales como Siemens Sinumeric, FANUC, Haas, Heidenhain o Mazak) y permite utilizar los mismos programas en distintas máquinas CNC de manera directa o con adaptaciones menores.

El G-code, también es conocido como RS-274 o lenguaje de programación G. Es usado principalmente en automatización y forma parte de la ingeniería o fabricación asistida por computadora. Los comandos G-Codes generalmente están acompañados de M-Codes.

En términos generales, G-code es un lenguaje mediante el cual se le indica a las CNC qué hacer y cómo hacerlo. Se trata de un lenguaje de programación vectorial mediante el que se describen acciones simples y entidades geométricas sencillas (básicamente segmentos de recta y arcos de circunsferencia) junto con sus parámetros de maquinado (velocidades de husillo y de avance de herramienta).

Existen diversas implementaciones del lenguaje. Además, los distintos fabricantes de máquinas CNC han añadido todo tipo de extensiones y variaciones al lenguaje, por lo que los operadores de las máquinas deben conocer las peculiaridades concretas que el fabricante ha previsto para su uso.

G-code comenzó siendo un tipo de lenguaje bastante limitado que carecía de estructuras como búcles, operadores condicionales y variables declaradas por el usuario. Sin embargo, las implementaciones más recientes de G-code sí que incluyen tales estructuras, creando un lenguaje algo más parecido a lo que podría ser un lenguaje de alto nivel.

A modo de ejemplo, se muestran los códigos de programación más utilizados en tornos de CNC de tecnoedu. Según el modelo, algunos de los códigos pueden estar deshabilitados.

Códigos Generales

- G00: Posicionamiento rápido (sin maquinar)
- G01: Interpolación lineal (maquinando)
- G02: Interpolación circular (horaria)
- G03: Interpolación circular (antihoraria)
- G04: Compás de espera
- G15: Programación en coordenadas polares
- G20: Comienzo de uso de unidades imperiales (pulgadas)
- G21: Comienzo de uso de unidades métricas
- G28: Volver al home de la máquina
- G40: Cancelar compensación de radio de curvatura de herramienta
- G41: Compensación de radio de herramienta a la izquierda
- G42: Compensación de radio de herramienta a la derecha
- G50: Cambio de escala

- G68: Rotación de coordenadas
- G73: Ciclos encajonados
- G74: Perforado con ciclo de giro antihorario para descargar virutas
- G76: Alesado fino
- G80: Cancelar ciclo encajonado
- G81: Taladrado
- G82: Taladrado con giro antihorario
- G83: Taladrado profundo con ciclos de retracción para retiro de viruta
- G90: Coordenadas absolutas
- G91: Coordenadas relativas
- G92: Desplazamiento del área de trabajo
- G94: Velocidad de corte expresada en avance por minuto
- G95: Velocidad de corte expresada en avance por revolución
- G98: Retorno al nivel inicial
- G99: Retorno al nivel R
- G107: Programación del 4º eje

Códigos Misceláneos

- M00: Parada
- M01: Parada opcional
- M02: Reset del programa
- M03: Hacer girar el husillo en sentido horario
- M04: Hacer girar el husillo en sentido antihorario
- M05: Frenar el husillo
- M06: Cambiar de herramienta
- M08: Abrir el paso del refrigerante
- M09: Cerrar el paso de los refrigerantes
- M10: Abrir mordazas
- M11: Cerrar mordazas
- M13: Hacer girar el husillo en sentido horario y abrir el paso de refrigerante
- M14: Hacer girar el husillo en sentido antihorario y abrir el paso de refrigerante
- M30: Finalizar programa y poner el puntero de ejecución en su inicio
- M38: Abrir la guarda
- M39: Cerrar la guarda
- M62: Activar salida auxiliar 1
- M67: Esperar hasta que la entrada 2 esté en ON
- M71: Activar el espejo en Y
- M80: Desactivar el espejo en X
- M81: Desactivar el espejo en Y
- M98: Llamada a subprograma
- M99: Retorno de subprograma

Referencias

- https://web.archive.org/web/20120226220922/http://www.tormach.com/machine_codes.html
- http://s3.cnccookbook.com/Downloads/eBook/CNCCookbookGCodeCourse.pdf
- https://tecnoedu.com/CNC/GM.php
- https://en.wikipedia.org/wiki/G-code

IV. Consideraciones a las comunicaciones por puerto serie

Es habitual que al tratar con comunicaciones serie se usen puertos POSIX (Linux / Unix).

En estos casos, los caracteres **\r** (retorno de carro, CR) y **\n** (nueva línea, LF) no se interpretan automáticamente si se configura el puerto en **modo raw** (binario), pero sí pueden ser transformados si ciertas banderas de **termios** están activas.

La estructura de configuración denominada termios tiene varios campos que afectan cómo se leen/escriben los caracteres.

```
struct termios {
    tcflag_t c_iflag; // input modes
    tcflag_t c_oflag; // output modes
    tcflag_t c_cflag; // control modes
    tcflag_t c_lflag; // local modes
};
```

Los que interesan son c_iflag, c_oflag y c_lflag, que afectan a \r y \n.

Flags de entrada (c_iflag — Input modes)

Estas flags definen cómo se **interpreta lo que llega del puerto**:

Flag	Efecto	Qué hace
`ICRNL`	Activa conversión `\r` → `\n`	Convierte cada `CR` recibido en `LF`
`INLCR`	Activa conversión `\n` → `\r`	Convierte cada `LF` recibido en `CR`
`IGNCR`	Ignora `\r`	Los `CR` se descartan (no se entregan al programa)

En modo "raw", normalmente se desactivan:

```
tty.c_iflag &= ~(ICRNL | INLCR | IGNCR);
```

Flags de salida (c_oflag — Output modes)

Estas flags definen cómo se **transforman los datos al enviar**:

Flag	Efecto	Qué hace
`ONLCR`	Convierte `\n` → `\r\n`	Al escribir un `LF`, se envían ambos
`OCRNL`	Convierte `\r` → `\n`	Al escribir `CR`, se cambia a `LF`
`ONOCR`	No envía `\r` al principio de una línea	
`ONLRET`	Considera `\r` como nueva línea también	

En modo "raw", normalmente se desactivan:

```
tty.c_oflag &= ~(OPOST | ONLCR | OCRNL);
```

Donde OPOST es la bandera general que activa el procesamiento de salida; si se quita, no se hace ninguna conversión.

Flags Locales (c_lflag — Local modes)

Afecta el modo "canónico" (lectura por líneas, con edición de entrada).

Flag	Efecto	Qué hace
`ICANON`	Modo de línea	Si está activado, `\n` y `\r` se usan como delimitadores
`ECHO`	Eco local	Imprime lo que se recibe, incluyendo `\n` / `\r`
`ISIG`	Señales (Ctrl-C, etc.)	No afecta directamente `\r` / `\n`, pero importante

En modo "raw" se suelen desactivar:

```
tty.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
```

Una forma de garantizar de manera explícita que el código opera en modo raw, sin traducciones de línea, lo cual implica que:

- Los \r y \n se envían exactamente como se los escriba.
- Lo que se lee, llega tal cual lo manda el dispositivo.

Es hacer:

```
tty.c_iflag &= ~(IXON | IXOFF | IXANY | ICRNL | INLCR | IGNCR);
tty.c_oflag &= ~(OPOST | ONLCR | OCRNL);
tty.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
```

Dado que el firmware usa librerías de tecnología Arduino, resulta importante recordar que:

Arduino envía con...	Terminadores finales	Notas
`Serial.print()`	Ninguno	texto puro
`Serial.println()`	`\r\n` (CR + LF)	igual que "Nueva línea" del Monitor serie
`Serial.write()`	Ninguno (binario)	no añade terminadores

Si bien hay muchas estrategias y algoritmos, una forma de limpiar los caracteres \r y \n de una cadena (con lógica de objetos) es definiendo la función clean dentro de alguna clase auxiliar:

```
std::string auxiliar::clean(const std::string& s) {
   std::string r = s;
   r.erase(std::remove(r.begin(), r.end(), '\r'), r.end());
   r.erase(std::remove(r.begin(), r.end(), '\n'), r.end());
   return r;
}
```

Y usándola luego, por ejemplo con:

```
std::string raw = serial.readString();
std::string msg = aux.clean(raw);
```